

个人工具网站平台

架构定义与实现说明文档

版本：V0.2

定位：个人自用、可扩展、可部署、支持 GitHub 模块安装

核心目标：先建立稳定主站平台，后续功能模块按统一模板开发、上传 GitHub，并通过后端管理页面一键安装、部署、启用、禁用、升级和回滚。

1. 项目定义

本项目是一个部署在个人服务器上的工具型网站平台。

网站不是开放式第三方插件市场，而是一个面向个人长期使用的私有模块平台。主站负责登录、用户管理、工具大厅、模块安装、模块部署、模块网关、网站内容编辑、运行状态监控和访问数据统计。各个功能模块作为独立 GitHub 仓库开发，按统一模块模板实现自己的前端、后端、数据库迁移和业务逻辑。

后续新增功能时，不应该修改主站核心代码。标准流程应该是：

1. 使用模块模板创建新模块
2. 开发模块前端、后端和数据库迁移
3. 将模块上传到 GitHub
4. 在主站后端管理页面填写 GitHub 仓库链接
5. 主站自动下载、校验、构建、部署模块
6. 模块出现在工具大厅
7. 用户点击后进入模块使用

2. 核心设计原则

2.1 主站稳定

主站是整个平台的稳定壳层，负责公共能力，不负责具体业务功能。

主站包括：

登录注册
当前用户信息
工具大厅

- 模块安装管理
- 模块部署管理
- 模块网关
- 后端管理平台
- 用户管理
- 运行状态监控
- 访问量统计
- 网站内容编辑
- 系统配置
- 备份与恢复

主站数据库在初始设计完成后，应尽量保持稳定。新增模块时，不应为了模块业务数据修改主站数据库。

2.2 模块独立

模块是独立功能应用，每个模块拥有自己的：

- GitHub 仓库
- module.yaml
- 前端工程
- 后端工程
- Dockerfile
- 数据库
- 数据库迁移
- 业务 API
- 业务表
- 版本信息
- 运行日志

模块不能直接访问主站数据库。模块只能通过主站转发的用户上下文识别当前用户。

2.3 统一登录，模块共享登录态

用户只需要在主站登录一次。

如果模块声明需要登录，则用户必须先登录主站，才能进入该模块。模块通过主站网关获得当前用户信息，并根据 `user_id` 隔离不同用户的数据。

如果模块声明不需要登录，则用户可以未登录直接进入并使用该模块。但是匿名使用不应留下任何持久化运行数据。每次打开模块时，都应该是初始状态。

2.4 模块数据库独立

每个模块使用自己的 PostgreSQL database。

采用方案 A：

同一个 PostgreSQL 实例
不同模块使用不同 database
不同模块使用不同 database user

示例：

```
PostgreSQL
├─ main_db
├─ module_todo_db
├─ module_decision_db
├─ module_journal_db
└─ module_map_footprint_db
```

主站数据库只保存平台级数据。模块业务数据全部保存在模块自己的数据库里。

2.5 后端管理平台必须存在

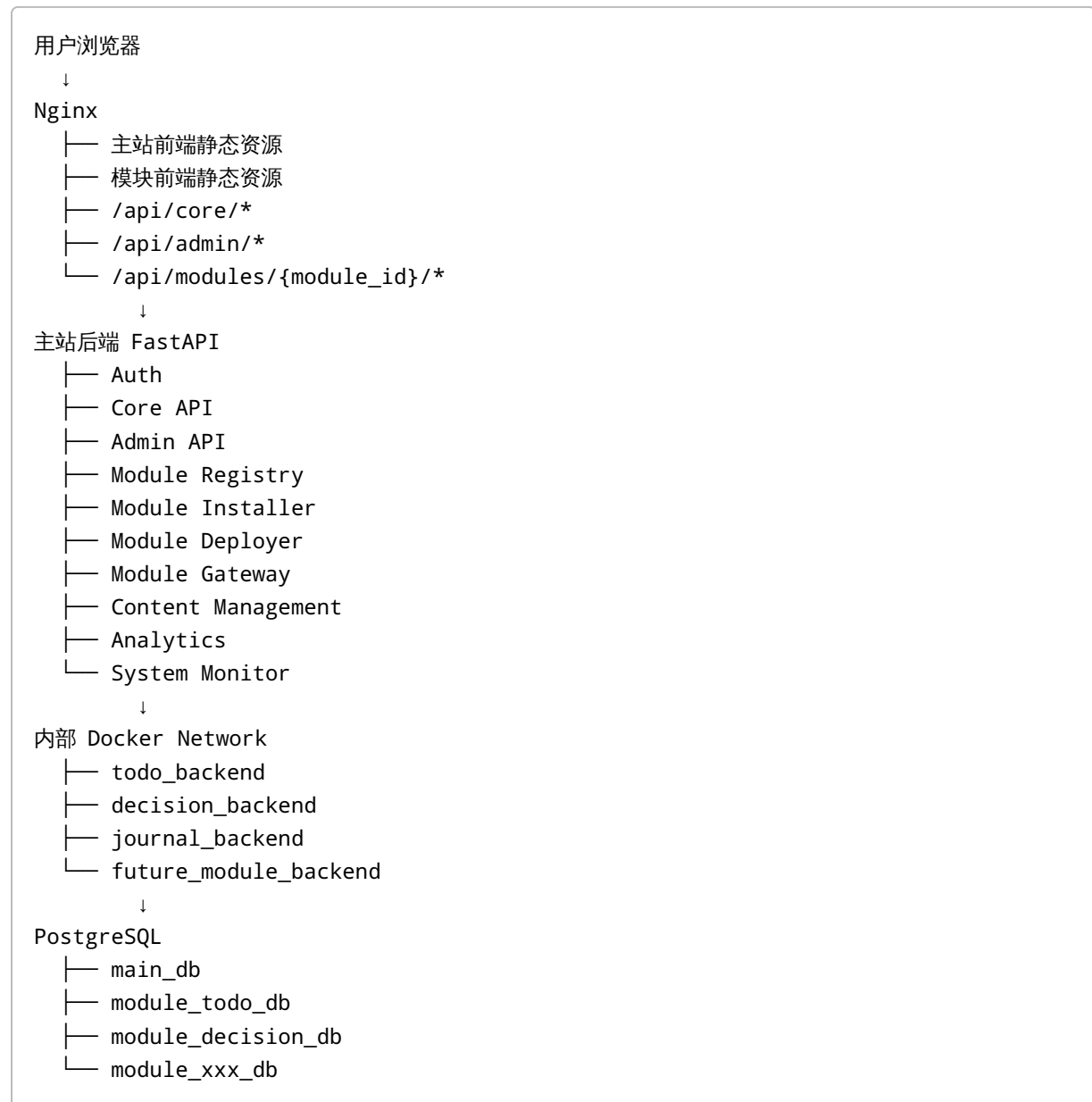
网站必须有一个后端管理页面，用于管理整个平台。

后端管理平台至少包括：

模块安装与部署
模块启用与禁用
模块升级与回滚
运行状态监控
用户管理
网站访问量数据
网站内容编辑
系统配置
运行日志
备份与恢复

3. 总体架构

推荐架构如下：



4. 技术选型

4.1 主站前端

推荐：

Vue 3
Vite
TypeScript
Pinia
Vue Router
Naive UI 或 Element Plus

主站前端负责：

主站 Shell
登录页
注册页
工具大厅
模块承载页
后端管理页面
用户中心
网站内容渲染

4.2 主站后端

推荐：

FastAPI
SQLAlchemy 2.x
Pydantic
Alembic
httpx
Docker SDK for Python
PostgreSQL

主站后端负责：

登录认证
用户信息
模块注册
模块安装
模块部署
模块网关
用户管理
访问统计

网站内容管理
运行状态检查

4.3 模块后端

模块后端推荐也使用 FastAPI，但不是强制要求。

模块只需要满足统一接口契约即可：

提供 Dockerfile
提供 /health
提供 /manifest
支持主站签名用户上下文
支持自己的数据库迁移

4.4 数据库

使用 PostgreSQL。

数据库分为：

main_db：主站数据库
module_xxx_db：模块数据库

主站数据库不保存模块业务数据。

4.5 部署

推荐：

Docker Compose
Nginx
PostgreSQL
Deploy Worker
模块容器

主站后端不直接执行高权限 Docker 操作。Docker 部署操作由独立的 Deploy Worker 负责。

5. 主站功能定义

5.1 主站 Shell

主站 Shell 是所有页面的统一外壳。

包括：

顶部导航栏
侧边栏
工具大厅入口
后端管理入口
用户头像
登录状态
退出登录
全局消息通知
全局 Loading
错误提示
内容区域

主站 Shell 不应该包含具体模块业务逻辑。

5.2 登录与用户系统

主站统一负责用户登录。

5.2.1 基础能力

注册
登录
退出
获取当前用户
修改个人资料
修改密码
头像设置
管理员角色
普通用户角色
禁用用户

5.2.2 推荐接口

```
POST    /api/auth/register
POST    /api/auth/login
POST    /api/auth/logout
POST    /api/auth/refresh
GET     /api/auth/me
PUT     /api/auth/profile
PUT     /api/auth/password
```

5.2.3 登录态存储

推荐：

```
Refresh Token : HttpOnly Cookie
Access Token  : 短期有效
```

不要长期把敏感 token 存在 LocalStorage。

5.3 工具大厅

工具大厅是所有模块的入口。

展示内容来自主站后端模块注册表，而不是写死在前端页面中。

5.3.1 工具大厅功能

```
展示已安装模块
展示模块图标、名称、简介、分类
支持搜索
支持分类筛选
支持收藏
支持置顶
支持最近使用
支持隐藏模块
支持登录要求提示
支持模块状态提示
```

5.3.2 模块卡片点击逻辑

点击模块卡片后，进入统一模块承载页：


```
/tools/{module_id}
```

例如：

```
/tools/todo  
/tools/decision-maker  
/tools/journal
```

5.4 模块承载页

所有模块都通过统一的 `ModuleHostPage` 打开。

推荐路径：

```
/tools/:moduleId
```

主站前端不再为每个模块单独写路由。

5.4.1 推荐展示方式

推荐使用：

主站 Shell + 模块运行窗口 + iframe 承载模块前端

原因：

- 模块可以独立构建
- 模块可以运行时安装
- 模块可以独立升级
- 模块前端不需要重新打包主站
- 模块样式不容易污染主站
- 模块出错不容易拖垮整个主站页面

6. 模块打开窗口的统一展示规范

推荐采用统一的“模块工作台窗口”方式。

整体结构：

```
主站 Shell
├── ModuleWorkspace
│   ├── ModuleHeader
│   ├── ModuleToolbar
│   ├── ModuleFrameContainer
│   │   └── iframe
│   └── ModuleStatusBar
```

6.1 ModuleHeader

模块顶部标题区域。

显示：

模块图标
模块名称
模块简介
当前版本
登录要求
运行状态
收藏按钮
更多操作按钮

示例：

[图标] Todo
任务管理模块 · v1.0.0 · 需要登录 · 运行中

6.2 ModuleToolbar

模块工具栏。

提供统一操作：

刷新模块
返回工具大厅
打开全屏

退出全屏
复制当前模块链接
查看模块信息
查看模块权限
管理员可见：重启模块
管理员可见：查看日志

6.3 ModuleFrameContainer

模块主体区域。

使用 iframe 加载模块前端：

```
/module-assets/{module_id}/current/index.html
```

iframe 样式建议：

宽度：100%
高度：calc(100vh - 主站顶部栏高度 - 模块标题栏高度)
边框：none
背景：透明或白色
圆角：跟随主站卡片风格

6.4 ModuleStatusBar

底部状态栏。

显示：

模块加载状态
最近一次健康检查时间
接口连接状态
当前用户状态
匿名模式提示

匿名模块应显示：

匿名模式：当前使用不会保存运行数据

需要登录模块应显示：

当前用户：用户名

6.5 模块展示模式

建议支持三种展示模式：

embedded：默认嵌入模式
fullscreen：全屏工作模式
new_window：新窗口打开模式

模块 manifest 中可以声明推荐模式：

```
display:
  mode: embedded
  allow_fullscreen: true
  allow_new_window: true
  min_width: 960
  min_height: 640
```

6.6 模块和主站通信

第一阶段建议尽量少通信。

必要通信通过 `postMessage` 实现。

主站可以发送：

```
theme_changed
locale_changed
refresh_requested
user_context_changed
```

模块可以发送：

```
module_ready
module_error
```

```
request_fullscreen
notify_success
notify_error
set_title
```

示例消息格式：

```
{
  "type": "module_ready",
  "module_id": "todo",
  "payload": {}
}
```

7. 模块系统定义

7.1 模块是什么

模块是一个独立功能单元。

每个模块至少包含：

```
module.yaml
frontend
backend
Dockerfile
数据库迁移
README
```

模块可以是：

```
需要登录的模块
无需登录的模块
有数据库的模块
无数据库的模块
纯前端模块
前后端完整模块
```

7.2 模块仓库标准结构

每个模块是一个独立 GitHub 仓库。

标准结构：

```
personal-tool-module-xxx/
├─ module.yaml
├─ frontend/
│   ├── package.json
│   ├── vite.config.ts
│   ├── index.html
│   └─ src/
│       ├── main.ts
│       ├── App.vue
│       ├── api.ts
│       └─ types.ts
├─ backend/
│   ├── Dockerfile
│   ├── pyproject.toml
│   └─ app/
│       ├── main.py
│       ├── core/
│       │   ├── auth.py
│       │   ├── database.py
│       │   └─ response.py
│       ├── api/
│       │   └─ router.py
│       ├── models/
│       ├── schemas/
│       └─ services/
├─ alembic/
│   ├── env.py
│   └─ versions/
├─ scripts/
│   ├── build_frontend.sh
│   ├── build_backend.sh
│   └─ migrate.sh
└─ README.md
```

7.3 module.yaml 定义

每个模块必须提供 `module.yaml`。

示例：

```
apiVersion: personal-tools/v1

id: todo
name: Todo
description: 任务管理模块
version: 1.0.0
category: productivity
icon: check-square

source:
  type: github

compatibility:
  main_api: ">=1.0.0 <2.0.0"
  module_sdk: ">=1.0.0 <2.0.0"

entry:
  type: iframe
  path: /tools/todo
  frontend_dist: frontend/dist

display:
  mode: embedded
  allow_fullscreen: true
  allow_new_window: true
  min_width: 960
  min_height: 640

backend:
  enabled: true
  type: fastapi
  dockerfile: backend/Dockerfile
  internal_port: 8000
  health_path: /health

database:
  enabled: true
  strategy: dedicated_postgres_database
  migrations: backend/alembic

auth:
  required: true
  user_context: signed_header
  anonymous_persistence: false
```

```
permissions:
  - todo:read
  - todo:write

features:
  supports_settings: true
  supports_upload: false
  supports_export: false
  supports_history: true
```

7.4 模块登录策略

模块可以自己声明是否需要登录。

配置：

```
auth:
  required: true
```

或者：

```
auth:
  required: false
  anonymous_persistence: false
```

7.4.1 需要登录的模块

如果 `auth.required = true`：

未登录用户不能进入模块
用户点击模块时跳转登录页
登录后回到原模块
模块 API 必须通过主站网关鉴权
模块后端可以获得当前用户信息
模块业务表必须按 `user_id` 隔离数据

7.4.2 不需要登录的模块

如果 `auth.required = false`：

未登录用户可以直接进入
模块可以正常使用
模块不能留下持久化运行数据
每次打开模块都应是初始状态
不能写入模块数据库
不能写入用户历史
不能写入主站用户设置
不能记录个人化业务状态

匿名模块允许保存的内容：

静态配置
公开内容
系统级访问统计
错误日志
健康检查日志

匿名模块不允许保存的内容：

用户输入的业务内容
使用历史
操作结果
个性化设置
可识别用户身份的数据

如果匿名模块需要临时状态，只能保存在：

前端内存
浏览器 SessionStorage

不建议保存在 LocalStorage，因为 LocalStorage 会跨会话保留。

8. 主站数据库设计

主站数据库使用 `main_db`。

主站数据库只保存平台级数据，不保存模块业务数据。

8.1 users

用户表。

```
CREATE TABLE users (  
  id UUID PRIMARY KEY,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  username VARCHAR(100) NOT NULL UNIQUE,  
  password_hash TEXT NOT NULL,  
  avatar_url TEXT,  
  role VARCHAR(50) NOT NULL DEFAULT 'user',  
  status VARCHAR(50) NOT NULL DEFAULT 'active',  
  created_at TIMESTAMP NOT NULL DEFAULT now(),  
  updated_at TIMESTAMP NOT NULL DEFAULT now()  
);
```

8.2 user_sessions

用户会话表。

```
CREATE TABLE user_sessions (  
  id UUID PRIMARY KEY,  
  user_id UUID NOT NULL REFERENCES users(id),  
  refresh_token_hash TEXT NOT NULL,  
  user_agent TEXT,  
  ip_address VARCHAR(100),  
  expires_at TIMESTAMP NOT NULL,  
  created_at TIMESTAMP NOT NULL DEFAULT now(),  
  revoked_at TIMESTAMP  
);
```

8.3 installed_modules

已安装模块表。

```
CREATE TABLE installed_modules (  
  id UUID PRIMARY KEY,  
  module_id VARCHAR(100) NOT NULL UNIQUE,  
  name VARCHAR(200) NOT NULL,  
  description TEXT,
```

```

category VARCHAR(100),
icon VARCHAR(100),

source_type VARCHAR(50) NOT NULL DEFAULT 'github',
source_url TEXT NOT NULL,
source_ref VARCHAR(200),
current_version VARCHAR(100),

entry_type VARCHAR(50) NOT NULL,
frontend_url TEXT,
api_prefix TEXT,
internal_backend_url TEXT,

auth_required BOOLEAN NOT NULL DEFAULT TRUE,
anonymous_persistence BOOLEAN NOT NULL DEFAULT FALSE,

display_config JSONB NOT NULL DEFAULT '{}',
manifest JSONB NOT NULL DEFAULT '{}',
runtime_config JSONB NOT NULL DEFAULT '{}',

status VARCHAR(50) NOT NULL DEFAULT 'installed',

created_at TIMESTAMP NOT NULL DEFAULT now(),
updated_at TIMESTAMP NOT NULL DEFAULT now()
);

```

8.4 module_versions

模块版本表。

```

CREATE TABLE module_versions (
  id UUID PRIMARY KEY,
  module_id VARCHAR(100) NOT NULL,
  version VARCHAR(100) NOT NULL,
  source_url TEXT NOT NULL,
  source_ref VARCHAR(200),
  commit_sha VARCHAR(100),

  release_path TEXT NOT NULL,
  frontend_path TEXT,
  docker_image TEXT,
  migration_version VARCHAR(100),

  status VARCHAR(50) NOT NULL DEFAULT 'available',

```

```
created_at TIMESTAMP NOT NULL DEFAULT now(),  
  
    UNIQUE(module_id, version)  
);
```

8.5 module_install_jobs

模块安装和部署任务表。

```
CREATE TABLE module_install_jobs (  
    id UUID PRIMARY KEY,  
    module_id VARCHAR(100),  
    job_type VARCHAR(50) NOT NULL,  
    status VARCHAR(50) NOT NULL DEFAULT 'pending',  
  
    source_url TEXT,  
    source_ref VARCHAR(200),  
  
    logs TEXT,  
    error_message TEXT,  
    result JSONB NOT NULL DEFAULT '{}',  
  
    created_by UUID REFERENCES users(id),  
    created_at TIMESTAMP NOT NULL DEFAULT now(),  
    started_at TIMESTAMP,  
    finished_at TIMESTAMP  
);
```

8.6 module_user_preferences

用户模块偏好表。

只保存主站层面的模块偏好，不保存模块业务数据。

```
CREATE TABLE module_user_preferences (  
    id UUID PRIMARY KEY,  
    user_id UUID NOT NULL REFERENCES users(id),  
    module_id VARCHAR(100) NOT NULL,  
  
    enabled BOOLEAN NOT NULL DEFAULT TRUE,  
    pinned BOOLEAN NOT NULL DEFAULT FALSE,
```

```
sort_order INTEGER DEFAULT 999,  
settings JSONB NOT NULL DEFAULT '{}',  
last_used_at TIMESTAMP,  
  
created_at TIMESTAMP NOT NULL DEFAULT now(),  
updated_at TIMESTAMP NOT NULL DEFAULT now(),  
  
UNIQUE(user_id, module_id)  
);
```

8.7 module_secrets

模块密钥表。

用于保存模块数据库密码、内部通信密钥等敏感信息。

实际实现时建议加密存储。

```
CREATE TABLE module_secrets (  
  id UUID PRIMARY KEY,  
  module_id VARCHAR(100) NOT NULL,  
  secret_key VARCHAR(100) NOT NULL,  
  encrypted_value TEXT NOT NULL,  
  created_at TIMESTAMP NOT NULL DEFAULT now(),  
  updated_at TIMESTAMP NOT NULL DEFAULT now(),  
  
  UNIQUE(module_id, secret_key)  
);
```

8.8 admin_audit_logs

管理员操作审计表。

```
CREATE TABLE admin_audit_logs (  
  id UUID PRIMARY KEY,  
  admin_user_id UUID REFERENCES users(id),  
  action VARCHAR(100) NOT NULL,  
  target_type VARCHAR(100),  
  target_id VARCHAR(100),  
  payload JSONB NOT NULL DEFAULT '{}',  
  ip_address VARCHAR(100),
```

```
    created_at TIMESTAMP NOT NULL DEFAULT now()  
);
```

8.9 site_contents

网站内容表。

用于后端管理平台编辑网站展示文字。

```
CREATE TABLE site_contents (  
    id UUID PRIMARY KEY,  
    content_key VARCHAR(200) NOT NULL UNIQUE,  
    title VARCHAR(200),  
    content_type VARCHAR(50) NOT NULL,  
    content_value JSONB NOT NULL DEFAULT '{}',  
    status VARCHAR(50) NOT NULL DEFAULT 'published',  
    version INTEGER NOT NULL DEFAULT 1,  
    updated_by UUID REFERENCES users(id),  
    created_at TIMESTAMP NOT NULL DEFAULT now(),  
    updated_at TIMESTAMP NOT NULL DEFAULT now()  
);
```

示例 content_key :

```
home.hero  
home.about  
home.footer  
tool_hub.title  
tool_hub.description  
login.title  
login.subtitle  
admin.dashboard.notice
```

8.10 page_views

网站访问记录表。

```
CREATE TABLE page_views (  
    id UUID PRIMARY KEY,  
    user_id UUID REFERENCES users(id),
```

```
session_id VARCHAR(100),
path TEXT NOT NULL,
module_id VARCHAR(100),
referrer TEXT,
user_agent TEXT,
ip_address VARCHAR(100),
created_at TIMESTAMP NOT NULL DEFAULT now()
);
```

8.11 system_metrics

系统指标表。

```
CREATE TABLE system_metrics (
  id UUID PRIMARY KEY,
  metric_key VARCHAR(100) NOT NULL,
  metric_value JSONB NOT NULL DEFAULT '{}',
  collected_at TIMESTAMP NOT NULL DEFAULT now()
);
```

9. 模块数据库实现方式

采用方案 A：

同一个 PostgreSQL 实例
每个模块一个独立 database
每个模块一个独立 database user

9.1 数据库命名规则

主站数据库：
main_db

模块数据库：
module_{module_id}_db

模块数据库用户：
module_{module_id}_user

例如：

```
module_todo_db  
module_todo_user
```

如果 module_id 包含短横线，应转换为下划线：

```
decision-maker → module_decision_maker_db
```

9.2 安装模块时自动创建数据库

Deploy Worker 在安装模块时执行：

```
CREATE USER module_todo_user WITH PASSWORD '随机生成密码';  
CREATE DATABASE module_todo_db OWNER module_todo_user;  
GRANT ALL PRIVILEGES ON DATABASE module_todo_db TO module_todo_user;
```

模块获得自己的数据库连接：

```
DATABASE_URL=postgresql://module_todo_user:password@postgres:5432/module_todo_db
```

9.3 模块数据库权限

模块数据库用户只能访问自己的数据库。

模块不能访问：

```
main_db  
其他模块数据库  
PostgreSQL 超级管理员权限
```

9.4 模块业务表必须包含 user_id

如果模块需要登录，则业务表必须包含 `user_id`。

示例：


```
CREATE TABLE todo_items (
  id UUID PRIMARY KEY,
  user_id UUID NOT NULL,

  title VARCHAR(200) NOT NULL,
  content TEXT,
  status VARCHAR(50) NOT NULL DEFAULT 'todo',
  priority VARCHAR(50),
  due_at TIMESTAMP,

  created_at TIMESTAMP NOT NULL DEFAULT now(),
  updated_at TIMESTAMP NOT NULL DEFAULT now()
);
```

由于模块数据库和主站数据库分离，不建议做跨数据库外键。模块通过主站签名用户上下文获得 `user_id`，并在查询时强制带上 `user_id` 条件。

正确查询方式：

```
item = (
  db.query(TodoItem)
  .filter(
    TodoItem.id == item_id,
    TodoItem.user_id == current_user.id,
  )
  .first()
)
```

错误查询方式：

```
item = db.query(TodoItem).filter(TodoItem.id == item_id).first()
```

9.5 不需要登录的模块数据库规则

如果模块不需要登录：

```
auth:
  required: false
  anonymous_persistence: false
```

则该模块不能持久化匿名用户的运行数据。

允许：

读取静态数据
读取公开配置
执行纯计算
前端临时状态
SessionStorage 临时状态

不允许：

写入用户输入
写入使用历史
写入匿名业务记录
写入长期偏好
写入可识别用户的数据

如果未来确实需要匿名持久化，必须单独设计匿名数据策略，不能默认开启。

10. 主站统一登录与模块用户上下文

10.1 总体流程

1. 用户登录主站
2. 主站设置 HttpOnly Cookie
3. 用户进入 /tools/{module_id}
4. 主站检查模块是否需要登录
5. 如果需要登录且用户未登录，则跳转登录页
6. 如果允许访问，则加载模块前端
7. 模块前端请求 /api/modules/{module_id}/xxx
8. 请求先进入主站 Module Gateway
9. 主站验证当前用户和模块权限
10. 主站向模块后端转发请求
11. 主站在请求头中附带签名用户上下文
12. 模块后端验证签名
13. 模块后端获得 current_user
14. 模块后端按 user_id 读写自己的数据库

10.2 用户上下文 Header

主站转发请求到模块后端时，添加：

```
X-PT-Module-Id: todo
X-PT-User-Context: base64url(json)
X-PT-User-Signature: hmac-sha256-signature
```

X-PT-User-Context 示例：

```
{
  "sub": "用户UUID",
  "username": "qingquan",
  "email": "xxx@example.com",
  "roles": ["user"],
  "module_id": "todo",
  "auth_required": true,
  "exp": 1730000000
}
```

模块后端只能信任主站签名过的 Header，不能信任前端直接传来的 `user_id`。

10.3 匿名访问 Header

如果模块不需要登录，主站可以转发匿名上下文：

```
{
  "sub": null,
  "anonymous": true,
  "module_id": "public-calculator",
  "auth_required": false,
  "persistence_allowed": false,
  "exp": 1730000000
}
```

模块收到：

```
anonymous = true
persistence_allowed = false
```

时，不允许写入业务数据。

11. Module Gateway 设计

主站后端提供统一模块网关。

外部访问路径：

```
/api/modules/{module_id}/{path:path}
```

内部转发路径：

```
http://module_{module_id}_backend:8000/{path}
```

示例：

```
GET /api/modules/todo/items
```

转发到：

```
http://module_todo_backend:8000/items
```

11.1 网关职责

Module Gateway 负责：

```
识别 module_id
检查模块是否安装
检查模块是否启用
检查模块运行状态
检查模块是否需要登录
检查当前用户登录状态
生成签名用户上下文
转发请求到模块后端
转发响应给前端
记录访问日志
处理模块错误
```

11.2 网关不负责

Module Gateway 不负责：

模块业务逻辑
模块数据库操作
模块内部权限细节
模块页面渲染

12. GitHub 模块安装机制

12.1 后端管理页面安装模块

管理员进入：

后台管理 → 模块管理 → 安装新模块

填写：

GitHub 仓库地址
分支 / Tag / Commit
是否立即部署
是否启用模块

示例：

<https://github.com/qingquanliao99/personal-tool-module-todo>

12.2 安装 API

POST /api/admin/modules/install

请求：

```
{
  "repo_url": "https://github.com/qingquanliao99/personal-tool-module-todo",
  "ref": "v1.0.0",
  "deploy_immediately": true,
  "enable_after_deploy": true
}
```

响应：

```
{
  "code": 0,
  "message": "install job created",
  "data": {
    "job_id": "job_xxx"
  }
}
```

12.3 安装任务状态查询

```
GET /api/admin/modules/install-jobs/{job_id}
```

安装状态：

```
pending
cloning
validating
building_frontend
building_backend
creating_database
migrating
starting_container
health_checking
success
failed
```

12.4 Deploy Worker 执行流程

部署操作由 Deploy Worker 执行。

流程：

1. 读取安装任务
2. clone GitHub 仓库到 staging 目录
3. 读取 module.yaml
4. 校验 module_id
5. 校验版本
6. 校验兼容性
7. 校验模块来源
8. 创建 release 目录
9. 构建模块前端
10. 构建模块后端 Docker image
11. 创建模块数据库和数据库用户
12. 写入模块 .env 文件
13. 执行模块 Alembic migration
14. 启动模块后端容器
15. 复制前端 dist 到模块静态资源目录
16. 执行健康检查
17. 写入 installed_modules
18. 写入 module_versions
19. 标记任务成功

12.5 模块目录结构

服务器目录建议：

```
/opt/personal-tool-site/
├─ docker-compose.yml
├─ .env
├─ main/
│   ├─ frontend/
│   └─ backend/
├─ modules/
│   ├─ staging/
│   │   └─ job_xxx/
│   ├─ installed/
│   │   └─ todo/
│   │       └─ current -> releases/1.0.0
│   │       └─ releases/
│   │           └─ 1.0.0/
│   │               └─ source/
│   └─ module.yaml
```

```
| | | | | frontend_dist/
| | | | | docker_image.txt
| | | | |
| | | | | assets/
| | | | | | todo/
| | | | | | | current -> ../../installed/todo/current/frontend_dist
| | | | | logs/
| | | | | | backups/
| | | secrets/
| | | | modules/
| | | | | todo.env
| | nginx/
| | | nginx.conf
```

13. 后端管理平台设计

后端管理平台路径建议：

```
/admin
```

仅管理员可访问。

13.1 管理首页 Dashboard

展示：

```
网站运行状态
当前在线用户数
今日访问量
今日模块调用量
已安装模块数量
运行中模块数量
异常模块数量
数据库状态
服务器 CPU 使用率
内存使用率
磁盘使用率
最近错误日志
最近部署任务
```


13.2 模块管理

功能：

- 查看已安装模块
- 安装新模块
- 部署模块
- 启用模块
- 禁用模块
- 重启模块
- 升级模块
- 回滚模块
- 删除模块
- 查看模块日志
- 查看模块健康状态
- 查看模块版本
- 查看模块数据库状态
- 备份模块数据
- 恢复模块数据

接口：

```
GET    /api/admin/modules
POST   /api/admin/modules/install
GET    /api/admin/modules/install-jobs/{job_id}
GET    /api/admin/modules/{module_id}
POST   /api/admin/modules/{module_id}/deploy
POST   /api/admin/modules/{module_id}/enable
POST   /api/admin/modules/{module_id}/disable
POST   /api/admin/modules/{module_id}/restart
POST   /api/admin/modules/{module_id}/upgrade
POST   /api/admin/modules/{module_id}/rollback
DELETE /api/admin/modules/{module_id}
GET    /api/admin/modules/{module_id}/logs
GET    /api/admin/modules/{module_id}/health
POST   /api/admin/modules/{module_id}/backup
POST   /api/admin/modules/{module_id}/restore
```

13.3 用户管理

功能：

查看用户列表
搜索用户
新增用户
禁用用户
启用用户
重置密码
修改用户角色
查看用户最近登录
查看用户模块使用情况
查看用户访问记录

接口：

```
GET    /api/admin/users
GET    /api/admin/users/{user_id}
POST   /api/admin/users
PUT    /api/admin/users/{user_id}
POST   /api/admin/users/{user_id}/disable
POST   /api/admin/users/{user_id}/enable
POST   /api/admin/users/{user_id}/reset-password
GET    /api/admin/users/{user_id}/activity
```

13.4 网站运行状态

展示：

主站后端状态
主站前端状态
PostgreSQL 状态
Nginx 状态
Deploy Worker 状态
模块容器状态
磁盘空间
CPU 使用率
内存使用率
最近异常
健康检查结果

接口：

```
GET /api/admin/system/status
GET /api/admin/system/health
GET /api/admin/system/metrics
GET /api/admin/system/logs
```

13.5 网站访问量数据

统计：

```
今日 PV
今日 UV
总 PV
总 UV
最近 7 天访问量
最近 30 天访问量
访问路径排行
模块访问排行
用户访问排行
设备类型
浏览器类型
来源页面
错误页面
```

接口：

```
GET /api/admin/analytics/overview
GET /api/admin/analytics/page-views
GET /api/admin/analytics/modules
GET /api/admin/analytics/users
GET /api/admin/analytics/referrers
```

前端每次路由变化时上报：

```
POST /api/core/analytics/page-view
```

请求：

```
{
  "path": "/tools/todo",
  "module_id": "todo",
```

```
"referrer": "/tools"  
}
```

13.6 网站内容编辑模块

网站内容编辑模块用于在后台管理平台编辑网站上展示的文字内容。

目标：

避免后期修改首页文案、说明文字、页脚文字时必须改代码

13.6.1 内容类型

支持：

```
plain_text : 纯文本  
rich_text : 富文本  
markdown : Markdown  
json_block : 结构化 JSON 内容  
link_list : 链接列表  
hero_block : 首页大标题区域  
faq_list : 常见问题  
notice : 公告
```

13.6.2 内容 key 设计

所有可编辑内容使用固定 key。

示例：

```
site.name  
site.description  
home.hero.title  
home.hero.subtitle  
home.hero.cta_text  
home.about  
home.notice  
tool_hub.title
```

```
tool_hub.description
login.title
login.subtitle
footer.text
footer.links
admin.dashboard.notice
```

13.6.3 内容渲染格式

前端通过内容 key 获取内容：

```
GET /api/core/site-contents?keys=home.hero.title,home.hero.subtitle
```

返回：

```
{
  "code": 0,
  "message": "ok",
  "data": {
    "home.hero.title": {
      "content_type": "plain_text",
      "content_value": {
        "text": "我的个人工具站"
      }
    },
    "home.hero.subtitle": {
      "content_type": "plain_text",
      "content_value": {
        "text": "集中管理我的生活、任务和记录工具"
      }
    }
  }
}
```

前端按预定义格式渲染：

```
<h1>{{ content['home.hero.title'].text }}</h1>
<p>{{ content['home.hero.subtitle'].text }}</p>
```

13.6.4 后台编辑接口

```
GET    /api/admin/site-contents
GET    /api/admin/site-contents/{content_key}
PUT    /api/admin/site-contents/{content_key}
POST   /api/admin/site-contents/{content_key}/publish
POST   /api/admin/site-contents/{content_key}/rollback
```

13.6.5 内容编辑原则

前端定义展示位置
后台编辑展示内容
后台不能任意注入脚本
富文本必须清洗 HTML
Markdown 渲染必须防 XSS
所有内容修改必须记录管理员和时间
重要内容支持版本回滚

14. API 路径规范

14.1 主站公开接口

```
/api/auth/*
/api/core/*
```

14.2 后台管理接口

```
/api/admin/*
```

14.3 模块业务接口

```
/api/modules/{module_id}/*
```

14.4 统一响应格式

所有接口返回：

```
{
  "code": 0,
  "message": "ok",
  "data": {}
}
```

错误响应：

```
{
  "code": 10001,
  "message": "unauthorized",
  "data": null
}
```

14.5 错误码建议

0：成功
10001：未登录
10002：无权限
10003：资源不存在
10004：参数错误
10005：资源冲突
20001：模块不存在
20002：模块未启用
20003：模块健康检查失败
20004：模块安装失败
20005：模块部署失败
30001：内容不存在
30002：内容格式错误
50000：服务器错误

15. 前端目录结构

主站前端建议：

```
frontend/
├─ src/
│   └─ app/
│       └─ App.vue
```

```
| router.ts
| layout/
|   MainLayout.vue
|   AdminLayout.vue
├ core/
|   api/
|     http.ts
|     types.ts
|   auth/
|     auth.store.ts
|     auth.guard.ts
|   modules/
|     module.api.ts
|     module.types.ts
|     module.store.ts
|   content/
|     content.api.ts
|     content.store.ts
|   analytics/
|     analytics.api.ts
├ pages/
|   HomePage.vue
|   LoginPage.vue
|   RegisterPage.vue
|   ToolHubPage.vue
|   ModuleHostPage.vue
|   ProfilePage.vue
├ admin/
|   AdminDashboardPage.vue
|   ModuleManagePage.vue
|   ModuleInstallPage.vue
|   UserManagePage.vue
|   SystemStatusPage.vue
|   AnalyticsPage.vue
|   SiteContentEditorPage.vue
└ components/
```

16. 后端目录结构

主站后端建议：

```
backend/
└ app/
```



```
|— main.py
|— core/
|   |— config.py
|   |— database.py
|   |— security.py
|   |— dependencies.py
|   |— response.py
|   |— exceptions.py
|   |— logging.py
|— modules/
|   |— auth/
|   |— users/
|   |— module_registry/
|   |— module_gateway/
|   |— module_installer/
|   |— module_deployer/
|   |— content/
|   |— analytics/
|   |— system_monitor/
|   |— admin/
|— worker/
|   |— deploy_worker.py
|— alembic/
|— tests/
```

17. Docker Compose 设计

主站基础服务：

```
services:
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf
      - ./modules/assets:/var/www/module-assets:ro
    depends_on:
      - backend

  backend:
    build: ./main/backend
    env_file:
```

```

    - .env
  depends_on:
    - postgres

  frontend:
    build: ./main/frontend

  postgres:
    image: postgres:16
    volumes:
      - ./data/postgres:/var/lib/postgresql/data

  deploy_worker:
    build: ./main/backend
    command: python -m app.worker.deploy_worker
    env_file:
      - .env
    volumes:
      - ./modules:/opt/personal-tool-site/modules
      - ./secrets:/opt/personal-tool-site/secrets
      - /var/run/docker.sock:/var/run/docker.sock
    depends_on:
      - postgres
      - backend

```

注意：

主站 backend 不挂载 docker.sock
 只有 deploy_worker 可以访问 docker.sock
 模块容器不暴露公网端口
 模块容器只加入内部 Docker network

18. Nginx 路由设计

/	→ 主站前端
/module-assets/	→ 模块前端静态资源
/api/auth/	→ 主站后端
/api/core/	→ 主站后端
/api/admin/	→ 主站后端
/api/modules/	→ 主站后端 Module Gateway
/uploads/	→ 上传资源

示例：

```
location /module-assets/ {  
    alias /var/www/module-assets/;  
    try_files $uri $uri/ =404;  
}  
  
location /api/ {  
    proxy_pass http://backend:8000;  
}
```

19. 安全设计

19.1 模块来源限制

只允许安装可信来源。

建议：

只允许自己的 GitHub 账号
只允许白名单组织
生产环境优先安装 tag 或 commit sha
不建议直接安装 main 分支

19.2 module.yaml 校验

必须校验：

module_id 格式
version 格式
路径不能包含 ../
frontend_dist 路径是否合法
Dockerfile 路径是否合法
health_path 是否合法
auth 配置是否完整
database 配置是否完整

19.3 容器隔离

模块容器不暴露公网端口
模块容器不能访问 Docker socket
模块容器只能访问自己的数据库
模块容器通过主站网关暴露 API

19.4 匿名模块安全

匿名模块不能留下运行数据。

主站网关应传递：

```
persistence_allowed = false
```

模块 SDK 应提供防御性封装。如果当前是匿名用户且不允许持久化，则写入接口应直接拒绝。

19.5 内容管理安全

网站内容编辑模块必须防止 XSS。

要求：

禁止后台内容注入 script
富文本渲染前清洗 HTML
Markdown 渲染前清洗 HTML
链接必须校验协议
管理员操作记录审计日志

20. 备份与恢复

必须备份：

```
main_db  
每个 module_xxx_db  
modules/installed  
modules/assets  
secrets/modules
```

```
uploads
nginx 配置
.env 配置
```

建议备份策略：

```
每日自动备份
保留最近 7 天
每周保留 1 份
每月保留 1 份
支持手动触发备份
支持模块级备份
支持主站级备份
```

后台管理页面应提供：

```
查看备份列表
手动备份
下载备份
恢复备份
删除旧备份
```

21. 模块模板开发规范

以后所有模块都按模板开发。

21.1 模块必须实现

```
module.yaml
frontend build
backend Dockerfile
/health
/manifest
数据库迁移
签名用户上下文解析
统一响应格式
```

21.2 模块不能做

不能访问主站数据库
不能保存用户密码
不能自行处理主站登录
不能信任前端传来的 `user_id`
不能暴露公网端口
不能修改主站文件
不能直接调用其他模块数据库

21.3 需要登录模块必须做

所有业务表包含 `user_id`
所有查询必须带 `user_id`
所有写入必须使用 `current_user.id`

21.4 不需要登录模块必须做

不保存运行数据
不写入业务历史
不保存匿名用户输入
每次打开保持初始状态

22. 推荐开发阶段

阶段 0：主站骨架

交付：

前端项目
后端项目
Docker Compose
Nginx
PostgreSQL
基础 README

阶段 1：登录与主站 Shell

交付：

注册
登录
退出
当前用户
主站布局
权限守卫

阶段 2：工具大厅与模块承载页

交付：

工具大厅
模块列表 API
/tools/:moduleId
ModuleHostPage
iframe 模块窗口

阶段 3：后端管理平台基础版

交付：

/admin
管理首页
模块管理页面
用户管理页面
系统状态页面

阶段 4：Module Gateway

交付：

/api/modules/{module_id}/{path}
模块启用检查
登录检查

签名用户上下文
请求转发

阶段 5：模块模板

交付：

personal-tool-module-template
示例 Todo 模块
独立数据库
Alembic migration
用户数据隔离
匿名模块示例

阶段 6：GitHub 安装器

交付：

填写 GitHub 链接
clone 仓库
读取 module.yaml
校验模块
写入安装任务
复制前端资源

阶段 7：一键部署器

交付：

构建前端
构建后端镜像
创建模块数据库
执行 migration
启动模块容器
健康检查
启用模块
失败回滚

阶段 8：网站内容编辑

交付：

内容 key 管理
内容编辑页面
内容发布
内容回滚
前端内容渲染

阶段 9：访问统计与运行监控

交付：

访问量统计
模块访问排行
用户访问数据
系统状态
模块健康检查
日志查看

23. 验收标准

23.1 主站验收

网站可以正常访问
用户可以注册、登录、退出
工具大厅可以展示模块
后端管理页面可以访问
管理员和普通用户权限区分正常

23.2 模块系统验收

可以通过 GitHub 链接安装模块
可以自动读取 `module.yaml`
可以构建模块前端
可以构建模块后端
可以创建模块独立数据库

可以执行模块 migration
可以启动模块容器
可以通过 /tools/{module_id} 打开模块
可以通过 /api/modules/{module_id} 调用模块 API

23.3 登录与数据隔离验收

需要登录的模块未登录不能进入
登录后可以进入需要登录的模块
模块可以获得当前用户信息
不同用户看到的数据互相隔离
模块不能访问主站数据库
模块不能访问其他模块数据库

23.4 匿名模块验收

不需要登录的模块未登录可以进入
匿名使用不保存业务数据
刷新或重新打开后恢复初始状态
匿名用户不能产生持久化业务历史

23.5 后台管理验收

可以查看运行状态
可以管理用户
可以查看访问量
可以安装模块
可以部署模块
可以启用和禁用模块
可以查看模块日志
可以编辑网站展示文字
前端可以按固定格式渲染后台编辑内容

24. 最终架构结论

本项目最终应定义为：

个人私有工具平台

而不是开放插件市场。

最终形态是：

主站是平台
模块是独立应用
GitHub 是模块分发来源
Deploy Worker 是安装部署执行器
Module Gateway 是统一鉴权和代理层
main_db 只保存平台数据
module_xxx_db 保存模块业务数据
用户只登录主站一次
模块通过主站签名用户上下文识别用户
后台管理平台负责部署、用户、状态、访问量和内容管理

这套架构的核心优势是：

主站长期稳定
模块可以独立开发
模块可以独立数据库
模块可以独立升级
模块可以独立回滚
新增功能不需要修改主站数据库
新增功能不需要重新设计登录
后续维护成本稳定

最终目标是让新增模块变成一个标准化流程：

复制模块模板
开发功能
上传 GitHub
后台填写链接
一键安装部署
工具大厅自动出现